Title: GNSS Lab Report

Lab 1: GNSS Positioning

Course:	AAE4203 — Guidance and Navigation
Group:	Group 19
Members:	**LI Xiao**; CHOY Tsz Him; XI Yilin; ZHOU Lianyi
Member IDs:	**24098902D**; 26132475D; 24108503D; 24114931D
Instructor:	Prof. Weisong WEN
Department:	Department of Aeronautical and Aviation Engineering
School:	The Hong Kong Polytechnic University
Date:	October 30, 2026

Abstract

This lab focused on GPS single-point positioning using GNSS data collected near Block X at PolyU. A u-blox ZED-F9P receiver recorded 768 epochs at 1 HZ over 12 minutes 47 seconds, including an initial static period followed by walking measuring. RINEX observations and navigation message were processed using MATLAB and MatRTKLIB. Positions were estimated in Python using equal-weighted least squares, elevation-dependent weighting and Huber robust weighting. The measuring data were checked using the example dataset. The elevation-weighted solution was compared with RTKLIB SPP and the receiver solution at 712 common epochs. For the first 60 seconds, horizontal RMS scatter about the mean was 30.06 m with elevation weighting and 26.26 m with Huber weighting. On the UrbanNav benchmark, the elevation-weighted solver returned 1437 solutions from 1534 observation epochs (93.67%).

Keywords: GNSS, single point positioning, weighted least squares, urban positioning, UrbanNav

---

1 Data Collection

GNSS data were collected near Block X at The Hong Kong Polytechnic University on 30 October 2026 using a u-blox ZED-F9P receiver with HPG 1.32 firmware. A multi-band GNSS antenna supplied with the laboratory kit was connected to the u-blox ZED-F9P receiver. Raw GNSS data were recorded using u-center 24.05. The experiment was conducted under sunny weather conditions in the semi-open campus environment near Block X. Surrounding buildings could obstruct or reflect GNSS signals. The recording began with an initial stationary period. During the subsequent walking portion, the GNSS antenna was held by hand while the group walked around Block X.The recording ran from 17:09:02 to 17:21:49, providing 768 observation epochs at 1 Hz over a 767-second interval. The receiver was initially stationary and was subsequently moved while the group walked around Block X. The first 60 seconds were selected as the confirmed static segment for repeatability analysis. The measurements were saved in UBX format, including RXM-RAWX, RXM-SFRBX and NAV-HPPOSECEF messages. The complete UBX file was converted to RINEX observation and navigation files using RTKLIB EX 2.5.0. The RINEX files were processed using MATLAB R2025a and MatRTKLIB to generate the inputs required by the Python SPP solver.



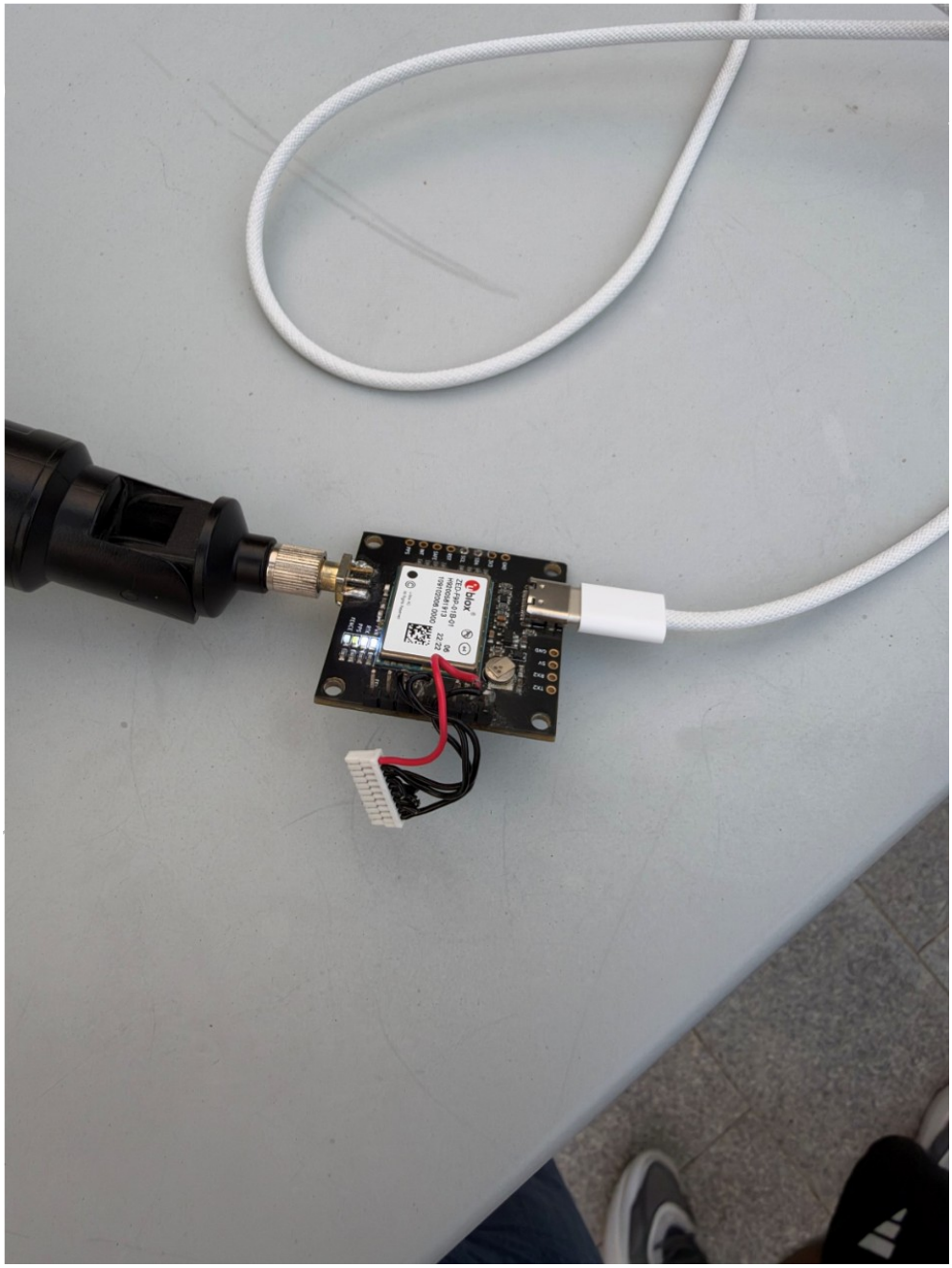


Figure 1: Data collection setting ups

Field	Value
Location	Vicinity of Block X
Duration	12 min 47 s, 1 Hz
Receiver	u-blox ZED-F9P (kit K09)

Table 1: Data collection summary

2 Raw Data Analysis

The raw GNSS observations were analysed using MATLAB and MatRTKLIB. Across the complete recording, 42 distinct satellites were observed: 10 GPS, 10 GLONASS, 12 Galileo and 10 BeiDou satellites. Figure 2 presents the primary-band carrier-to-noise density ratio (C/N_0) and tracking availability of individual satellites throughout the recording.

For records containing valid primary-band pseudoranges, the median C/N_0 values were 35, 33, 34 and 35 dB-Hz for GPS, GLONASS, Galileo and BeiDou, respectively. Several satellites exhibited intermittent tracking gaps and substantial variations in C/N_0, showing that signal reception was not uniform throughout the experiment.

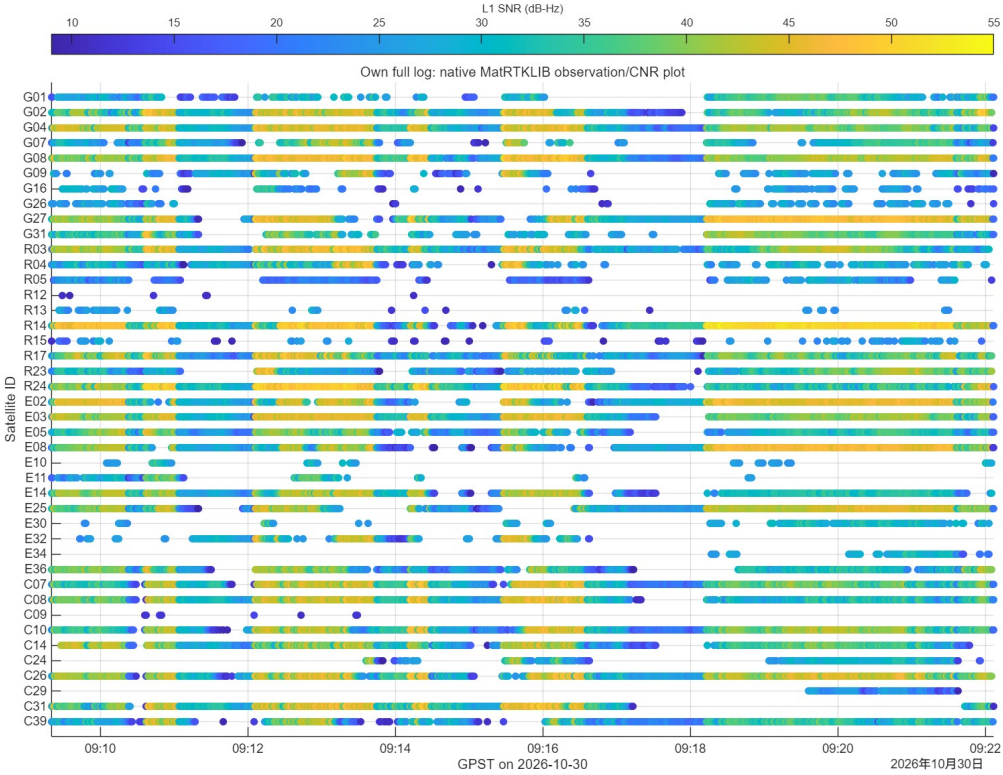


Figure 2: Primary-band C/N_0 and tracking availability of the self-collected GNSS observations, generated using MatRTKLIB. Time is expressed in GPST.

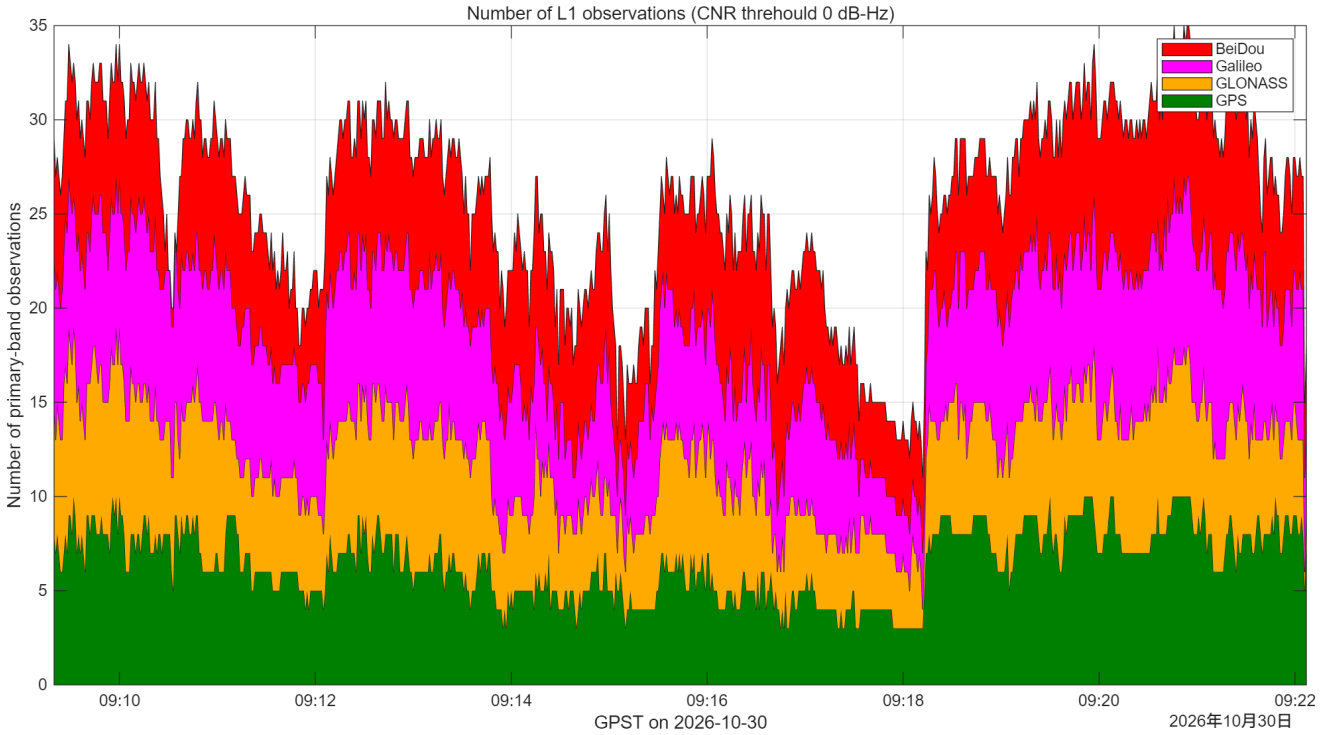


Figure 3: Primary-band observation counts by constellation for the data collected near Block X.

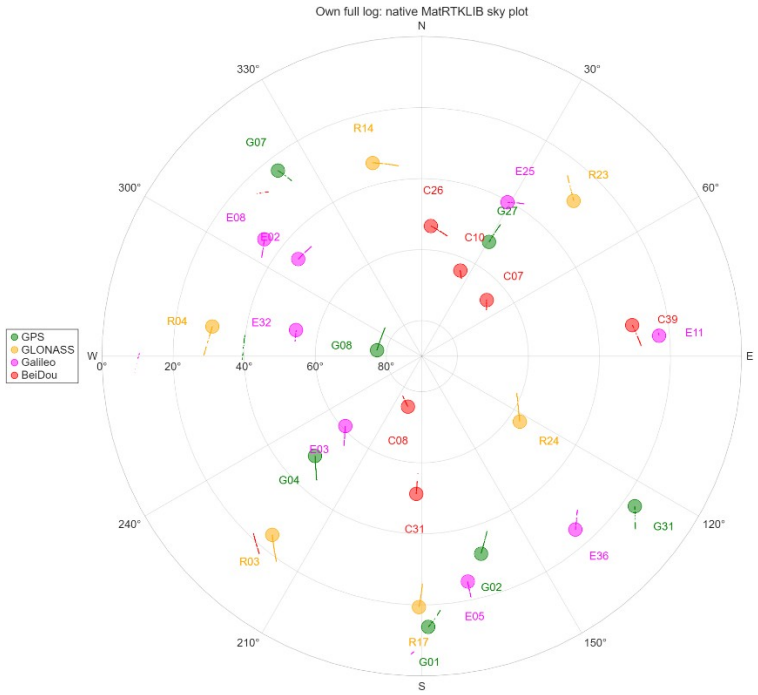


Figure 4: Satellite azimuth and elevation distribution for the data collected near Block X.

3 SPP Algorithm (Least Squares)

Single point positioning (SPP) estimates the receiver's ECEF coordinates and clock bias from GNSS pseudorange observations. The GPS L1 implementation uses at least four valid satellite

measurements. Satellite clock, ionospheric, tropospheric and group-delay corrections are applied before positioning. Equation (1) represents the simplified observation model, where ρ_i is the corrected pseudorange, x is the receiver position, s_i is the satellite position, c is the speed of light, and Δt is the receiver clock offset [1, 2]. The implementation also accounts for Earth rotation during range prediction.

$$\rho_i = \| x - s_i \| + c\,\Delta t + \varepsilon_i, \tag{1}$$

The observation equations are linearised around the current state estimate, giving the following weighted least-squares update:

$$\delta = (H^{\top} W H)^{-1} H^{\top} W v, \tag{2}$$

Here, H is the design matrix, v contains the observed-minus-predicted pseudorange residuals, and δ is the correction to the receiver position and clock bias. Three weighting configurations were tested: equal weights, elevation-dependent weights, and Huber robust reweighting. The elevation-dependent weights are based on the squared sine of satellite elevation. Huber reweighting reduces the influence of large normalised residuals using a threshold of 2.5 and an a priori zenith pseudorange scale of 3 m. Iterations terminate when the state-update norm falls below 10^{-4} m, with a maximum of 80 iterations per stage.

4 Code Listing

Listing 1 presents the core Python implementation used for the GPS-SPP experiments. The complete implementation and input-processing scripts are included in the accompanying code package.Listing 1 presents the core Python implementation used for the GPS-SPP experiments. The complete implementation and input-processing scripts are included in the accompanying code package.

```
# Report Code Listing: extracted from the actual analysis implementation.
# Dependency helper is provided in 90_Complete_Results/code/tools/legacy_solver.py.
import numpy as np
from tools.legacy_solver import ecef_to_lla
C = 299792458.0
OMEGA = 7.2921151467e-5

def rotation(xyz):
    lat,lon,h=ecef_to_lla(*xyz); a,b=np.radians([lat,lon])
    return np.array([
        [-np.sin(b), np.cos(b), 0],
        [-np.sin(a)*np.cos(b), -np.sin(a)*np.sin(b), np.cos(a)],
        [np.cos(a)*np.cos(b), np.cos(a)*np.sin(b), np.sin(a)]
    ])
def model(sp,x):
    d=sp-x[:3]; r=np.linalg.norm(d,axis=1)
```

```
    sag=OMEGA/C*(sp[:,0]*x[1]-sp[:,1]*x[0])
    H=np.column_stack([-d/r[:,None],np.ones(len(sp))])
    H[:,0]+=-OMEGA/C*sp[:,1]; H[:,1]+=OMEGA/C*sp[:,0]
    sine = (d @ rotation(x[:3])[2] / r
            if np.linalg.norm(x[:3]) > 6e6 else np.ones(len(sp)))
    return r+sag+x[3],H,sine

def solve(sp,corrected,seed,method):
    x=np.r_[seed,0.]; base=None
    # A converged elevation WLS initializes the Huber experiment.
    stages = (['elevation', 'huber']
              if method == 'huber' else [method])
    for stage in stages:
        converged=False
        for it in range(80):
            pred,H,sine=model(sp,x); v=corrected-pred
            w = (np.ones(len(sp)) if stage == 'uniform'
                 else np.maximum(sine, .05)**2)
            if stage=='huber':
                # Fixed a priori zenith pseudorange scale, 3 m; threshold 2.5 sigma.
                z=np.abs(v)*np.sqrt(w)/3.
                w*=np.minimum(1.,2.5/np.maximum(z,1e-12))
            a=np.sqrt(w)
            dx, _, rank, _ = np.linalg.lstsq(
                H * a[:, None], v * a, rcond=None
            )
            if rank<4: raise ValueError('rank deficient GPS geometry')
            x+=dx
            if np.linalg.norm(dx)<1e-4: converged=True;break
        if not converged: raise ValueError(stage+' iteration did not converge')
    if (not np.isfinite(x).all()
            or not 6.30e6 < np.linalg.norm(x[:3]) < 6.42e6):
        raise ValueError('not an Earth surface solution')
    pred,H,sine=model(sp,x); v=corrected-pred
    R=rotation(x[:3]); Q=np.linalg.inv(H.T@H); QE=R@Q[:3,:3]@R.T
    return (
        x, v,
        float(np.sqrt(QE[0, 0] + QE[1, 1])),
        float(np.sqrt(np.trace(Q[:3, :3]))),
        (int(np.sum(np.abs(v)*np.maximum(sine, .05)/3 > 2.5))
         if method == 'huber' else 0)
    )
```

Listing 1: Core GPS-SPP implementation with equal, elevation-

dependent and Huber weights.

5 Own-Data Validation

Input checks confirmed a complete UBX-derived observation record containing 768 epochs at 1 Hz. The elevation-weighted GPS-SPP solution was compared with a newly computed RTKLIB GPS-only single-point solution and the receiver NAV-HPPOSECEF solution. A total of 712 common epochs were matched by GPS time. Table 2 covers the full recording, including walking periods. Figure 3 uses only the first 60 s (60 epochs), confirmed as stationary, with positions expressed relative to their mean in a local East–North–Up (ENU) frame.

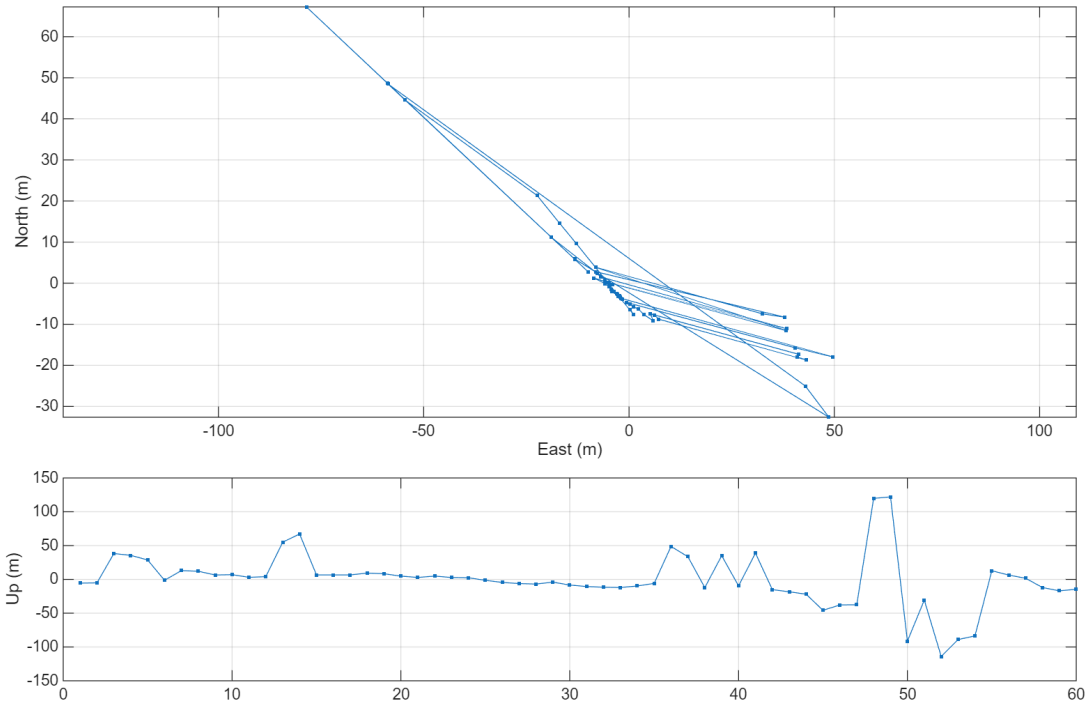


Figure 5: First-60-second GPS-SPP positions relative to their mean: East–North positions (top) and Up component (bottom).

Table 2: Differences between the GPS-SPP solution and reference solutions for the self-collected dataset.

Statistic	SPP − RTKPOST	SPP − receiver
Mean E / N / U (m)	0.187 / −0.898 / 2.138	7.088 / −2.422 / 13.939
Std E / N / U (m)	1.654 / 2.614 / 9.535	24.428 / 15.392 / 44.449
2D RMS difference (m)	3.224	29.809
Epochs compared	712	712

6 Benchmark Results

Three GPS-L1 versions were tested on the provided example dataset before processing the

UrbanNav benchmark. Version v1 used equal weights. Version v2 introduced elevation-dependent weights to reduce the contribution of low-elevation measurements. Version v3 added Huber robust weights to limit the influence of large residuals. The input data and measurement-screening thresholds were held fixed across the three versions.

Table 3: Local processing diagnostics for the UrbanNav benchmark.

Version	Change tested	Solved epochs	Local availability (%)	Mean post-fit pseudorange RMS (m)
v1	GPS L1, equal weights	1437	93.677	3.434
v2	v1 + elevation-dependent weights	1437	93.677	4.038
v3	v2 + Huber robust weights	1419	92.503	4.300

Local availability is calculated relative to the 1534 observation epochs. Mean post-fit pseudorange RMS is averaged over each version's solved epochs and describes measurement fit, rather than positioning accuracy. The Huber variant produced 18 fewer solutions than the other versions. These diagnostics alone do not establish which method gives the most accurate positions.

7 AI-Use Statement

OpenAI Codex was used to interpret the laboratory requirements, assist with developing and debugging the GNSS processing scripts. Verification included tests on the provided example dataset, a synthetic-state test, time and unit checks, CSV-format checks, and comparisons with a recomputed RTKLIB GPS-only single-point solution. The GNSS analysis figures selected for this report were generated using native plotting functions in the course-provided MatRTKLIB. Key prompts and the corresponding verification steps are listed in Appendix B.

8 Page Limits and Submission

- Report: max 10 pages; code listings in the report: max 2 pages (include only key parts).
- GitHub repository: raw data, documented Python scripts and README.md; it must reproduce both submitted CSV files.
- Submit the PDF report on Blackboard and give the GitHub link in the report; upload GXX_benchmark.csv and GXX_owndata.csv to the TA's Google Drive folder.
- Deadline: announced on Blackboard.
- https://github.com/lix09797-png/AAE4203-G19-GNSS-Lab

References

[1] P. Misra and P. Enge, Global Positioning System: Signals, Measurements, and Performance, 2nd ed., Ganga-Jamuna Press, 2011.
[2] P. J. G. Teunissen and O. Montenbruck (Eds.), Springer Handbook of Global Navigation Satellite Systems, Springer, 2017.
[3] L.-T. Hsu et al., "UrbanNav: An open-sourced multisensory dataset for benchmarking positioning algorithms designed for urban areas," in Proc. ION GNSS+ 2021, pp. 226–256.

A Figure and Table Examples

Use meaningful captions and labels; reference figures/tables in text (e.g., Fig. 1, Table 1).

B Key AI Prompts

Prompt:

1. "These are my collected data and the relevant course guides. Please help me complete the tasks according to the instructor's requirements, using the lab report requirements as the main reference. Ask me if anything about the files is unclear."

2. "Upload the required materials to my GitHub account according to the instructor's requirements."

AI-assisted work: The AI reviewed the laboratory tutorial, lecture slides, report template and supplied code, then identified the required processing steps and report materials. The AI organised and uploaded the original GNSS data, processing inputs, Python and MATLAB source code, two trajectory CSV files, native MATLAB figures, validation records and README instructions.

Verification method: The proposed tasks were cross-checked against the required report sections. Instructions in the documents were distinguished from information supplied by the group. Equipment details and field conditions were checked against available records or user confirmation, rather than assumed from the template examples.